

Mundo dos Blocos de Tamanho Variável: Planejamento com SAT nas Situações 1, 2 e 3

Fundamentos de Inteligência Artificial (IComp/UFAM)

Gabriel Menezes Rodrigues (gabriel.menezes@icomp.ufam.edu.br)

Luiz Felipe Gonzaga do Carmo (luiz.carmo@icomp.ufam.edu.br)

Joshua Tadeu Bastos Pereira (joshua.pereira@icomp.ufam.edu.br)

Romário Roberto de Pádua Andrade (romario.andrade@icomp.ufam.edu.br)

Outubro de 2026

Sumário

1	Introdução ao Problema	2
2	Descrição Formal do Mundo dos Blocos de Tamanho Variado	3
2.1	Domínio	3
2.2	Predicados e axiomas	3
2.3	A ação $move(b, y, p)$	4
2.4	Elementos novos e as ações associadas	5
2.5	Ajustes em relação ao mundo clássico	5
2.6	Execução manual	6
2.6.1	Situação 1: $S_0 \rightarrow S_{f4}$	6
2.6.2	Situação 2: $S_0 \rightarrow S_5$	7
2.6.3	Situação 3: $S_0 \rightarrow S_7$	8
2.7	Ordem parcial	11
2.7.1	Situação 1	11
2.7.2	Situação 2	13
2.7.3	Situação 3	13
2.7.4	Ordem parcial no SAT	15
3	Codificação CNF	15
3.1	Variáveis proposicionais	16
3.2	Grupos de cláusulas	16
3.3	A relação <i>on</i> não vira variável	17
4	Exemplos dos 3 cenários codificados para CNF em LP	17
4.1	Situação 1: S_0 e meta S_{f4}	17
4.2	Situação 1: exemplos de cláusulas geradas	17
4.3	Situação 2: S_0 e meta S_5	18
4.4	Situação 3: S_0 e meta S_7	18
5	Mapeamento: Descrição Formal $\rightarrow$ Código	19
6	Execução Passo a Passo	19

7	Interpretação da Saída do SAT Solver	20
7.1	De inteiros para o plano	20
7.2	Comparação: plano manual $\times$ plano do SAT	21
7.3	Resumo	21

1 Introdução ao Problema

No Mundo dos Blocos clássico todos os blocos têm o mesmo tamanho. Para descrever um estado basta dizer qual bloco está em cima de qual ($on(x, y)$) e quais blocos não têm nada em cima ($clr(x)$). Neste trabalho os blocos têm comprimentos diferentes (1, 2 e 3 unidades) e a mesa tem 6 espaços, chamados de *slots*. Por isso também é preciso saber em que posição da mesa cada bloco está. Isso traz situações que não aparecem no caso clássico:

- espaços vazios entre os blocos e embaixo de blocos que ficam suspensos;
- pontes, quando um bloco comprido fica apoiado em dois blocos menores separados;
- equilíbrio, porque um bloco maior que o de baixo só fica parado se tiver apoio suficiente;
- movimentos laterais, porque o mesmo bloco pode ficar em mais de uma posição em cima do mesmo apoio.

O problema de planejamento é o seguinte: temos um estado inicial e um estado meta, e queremos uma sequência de ações (um plano) que leve do primeiro ao segundo. O trabalho foi feito em três etapas:

1. descrevemos o mundo em Lógica de Primeira Ordem (LPO), com os predicados, os axiomas e a ação *move* com suas pré-condições e efeitos (Seção 2);
2. resolvemos cada cenário à mão e fizemos a análise de ordem parcial (Seções 2.6 e 2.7);
3. passamos a descrição para Lógica Proposicional em CNF, com um número fixo de H passos, resolvemos com o MiniSat e interpretamos a resposta (Seções 3 a 7).

O que foi resolvido. Resolvemos os três cenários pedidos no enunciado (Figura 1):

Cenário	Estado inicial	Meta	Plano ótimo
Situação 1	S_0	S_{f4} (escolhida entre $S_{f1}-S_{f4}$)	4 ações
Situação 2	S_0 da Situação 2	S_5	5 ações
Situação 3	S_0 da Situação 3 (igual ao da Situação 1)	S_7	6 ações

- Na Situação 1 escolhemos a meta S_{f4} porque ela tem o plano mais curto. Para S_{f1} são necessárias 9 ações, e para S_{f2} e S_{f3} são 10.
- O enunciado pede “ S_0 até S_5 ” e “ S_0 até S_7 ”. Entendemos que S_5 e S_7 são os estados finais das Situações 2 e 3, e que cada uma começa no seu próprio S_0 . Se o S_0 do item 2 for o da Situação 1, a meta S_5 fica igual a S_{f2} e o menor plano tem 10 ações. Também testamos esse caso no SAT.
- Como o enunciado permite, usamos um chatbot de IA (Claude) para ajudar a escrever o código Python e a montar os planos. Conferimos todos os planos passo a passo e também com o SAT (Seção 7).

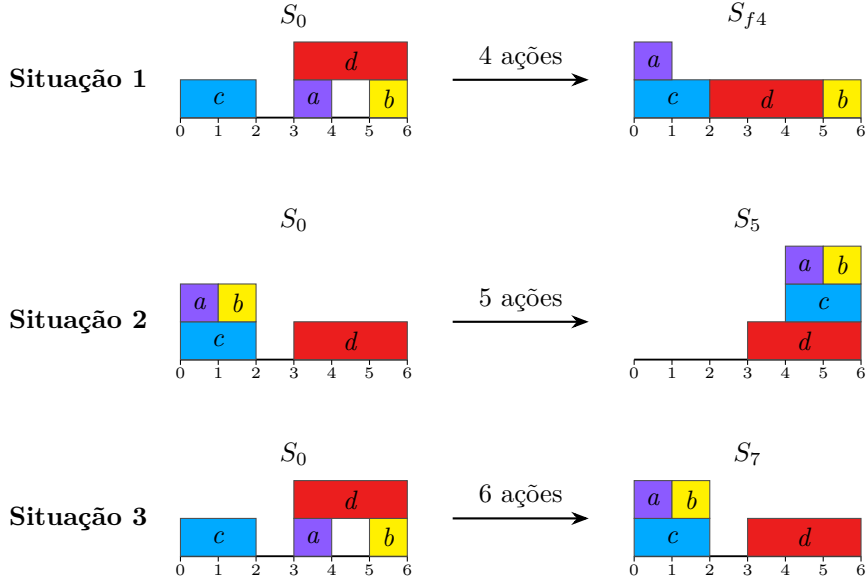


Figura 1: Os três problemas resolvidos.

Notação. A mesa é T . O número de passos do plano (o horizonte) é H (no código, `HORIZON`).

2 Descrição Formal do Mundo dos Blocos de Tamanho Variado

2.1 Domínio

- Blocos $\mathcal{B} = \{a, b, c, d\}$ com comprimento ℓ : $\ell(a) = \ell(b) = 1$, $\ell(c) = 2$, $\ell(d) = 3$.
- Pontos do eixo $\mathcal{X} = \{0, \dots, 6\}$ e slots $s_i = [i, i + 1]$, $i \in \{0, \dots, 5\}$ (6 slots, não 7).
- Níveis $\mathcal{L} = \{0, 1, 2, 3\}$ (0 = sobre a mesa) e instantes $t \in \{0, \dots, H\}$.
- Um bloco b que começa no ponto p ocupa $span(b, p) = \{p, \dots, p + \ell(b) - 1\}$, com $p \in \{0, \dots, 6 - \ell(b)\}$.
- $overlap(b, p, y, q) \leftrightarrow span(b, p) \cap span(y, q) \neq \emptyset$.

2.2 Predicados e axiomas

Predicado	Significado	Tipo
$at(b, p, t)$	b começa no ponto p no instante t	básico
$lev(b, l, t)$	b está no nível l	básico
$cov(b, s, l, t)$	b cobre o slot s no nível l	calculado
$livre(s, l, t)$	nenhum bloco cobre s no nível l	calculado
$esp(b, s, t)$	(novo) há espaço livre em cima de b , no slot s	calculado
$clr(b, t)$	não há nada em cima de b (espaço livre em todos os slots)	calculado
$on(b, y, t)$	b está apoiado em $y \in \mathcal{B} \cup \{T\}$	calculado

Para saber como o mundo está em um instante basta guardar at e lev de cada bloco. Por isso chamamos esses dois de básicos. Os outros predicados são calculados a partir deles, usando as

regras abaixo (elas valem para todo b, y, p, s, l, t):

$$\forall b, t \exists! p \text{ at}(b, p, t) \quad \forall b, t \exists! l \text{ lev}(b, l, t) \quad (\text{A1 unicidade})$$

$$\text{cov}(b, s, l, t) \leftrightarrow \text{lev}(b, l, t) \wedge \exists p (\text{at}(b, p, t) \wedge s \in \text{span}(b, p)) \quad (\text{A2})$$

$$b \neq b' \rightarrow \neg(\text{cov}(b, s, l, t) \wedge \text{cov}(b', s, l, t)) \quad (\text{A3 exclusão})$$

$$\text{livre}(s, l, t) \leftrightarrow \neg \exists b \text{ cov}(b, s, l, t) \quad (\text{A4})$$

$$\text{esp}(b, s, t) \leftrightarrow \exists l (\text{cov}(b, s, l, t) \wedge \text{livre}(s, l + 1, t)) \quad (\text{A5a espaço})$$

$$\text{clr}(b, t) \leftrightarrow \forall s, l (\text{cov}(b, s, l, t) \rightarrow \text{esp}(b, s, t)) \quad (\text{A5b clear})$$

$$\text{on}(b, \top, t) \leftrightarrow \text{lev}(b, 0, t) \quad (\text{A6})$$

$$\text{on}(b, y, t) \leftrightarrow \exists p, q, l \text{ at}(b, p, t) \wedge \text{at}(y, q, t) \wedge \text{lev}(b, l, t) \wedge \text{lev}(y, l - 1, t) \wedge \text{overlap}(b, p, y, q)$$

$$\text{at}(b, p, t) \wedge \text{lev}(b, l, t) \wedge l > 0 \rightarrow \left| \{s \in \text{span}(b, p) : \neg \text{livre}(s, l - 1, t)\} \right| \geq \left\lceil \frac{\ell(b)}{2} \right\rceil \quad (\text{A7 estabilidade})$$

O predicado *esp* é novo. Ele funciona como um *clr* que olha um slot de cada vez: $\text{esp}(b, s)$ é verdadeiro quando não tem nada em cima de b no slot s . Já o $\text{clr}(b)$ só é verdadeiro quando todos os slots de b estão sem nada em cima (A5b). No último nível ($l = 3$) não cabe nenhum bloco acima, então tratamos $\text{livre}(s, 4, t)$ como sempre verdadeiro.

A regra A7 trata do equilíbrio. Um bloco que está em cima de outro precisa ter pelo menos metade dos seus slots apoiados. Os blocos a e b precisam de 1 slot apoiado. O bloco c também precisa de 1, então pode ficar com metade para fora. O bloco d precisa de 2. É por isso que a ponte de S_0 é válida: d está apoiado em a e em b , mesmo com o slot s_4 vazio embaixo dele.

2.3 A ação $\text{move}(b, y, p)$

“Mover o bloco b para cima de y (ou da mesa $\top$), começando no ponto p .” Chamamos de (p_0, l_0) a posição atual de b e de l^* o nível em que ele vai ficar: $l^* = 0$ se $y = \top$ e $l^* = \text{lev}(y) + 1$ se y é bloco.

Pré-condições. A ação só pode ser executada se todas as condições abaixo forem verdadeiras:

P1 $\text{clr}(b)$: não pode ter nada em cima de b .

P2 Se y é um bloco, b tem que ficar de fato em cima de y , isto é, pelo menos um slot de b tem que cair sobre um slot de y ($\text{overlap}(b, p, y, q)$, com $\text{at}(y, q)$). Além disso, nesses slots tem que haver espaço livre em cima de y ($\text{esp}(y, s)$). Se o próprio b já estiver em um desses slots, ele não conta como ocupado. Esta condição entra no lugar do $\text{clr}(y)$ do mundo clássico (Seção 2.5).

P3 O destino tem que ser diferente do lugar atual: $(p, l^*) \neq (p_0, l_0)$.

P4 Todos os slots de destino, $s \in \text{span}(b, p)$, têm que estar livres no nível l^* (sem contar o próprio b).

P5 Regra da sombra: não pode ter nenhum bloco acima dos slots de destino, em nenhum nível maior que l^* . Ou seja, não dá para colocar um bloco embaixo de outro.

P6 $l^* \leq 3$, para não passar do último nível.

A estabilidade (A7) não aparece como pré-condição porque ela já é uma regra que vale para todos os estados. Se o movimento deixasse o bloco sem apoio suficiente, o estado de chegada não seria permitido.

Efeitos. Usamos o estilo STRIPS, com uma lista do que passa a valer depois da ação (ADD) e uma lista do que deixa de valer (DEL). Aqui z é o bloco onde b estava apoiado e y' é o bloco onde b fica apoiado depois.

Passa a valer (ADD):

- a nova posição de b : $at(b, p)$ e $lev(b, l^*)$;
- os slots que b passa a cobrir: $cov(b, s, l^*)$ para $s \in span(b, p)$;
- os novos apoios: $on(b, y')$ (podem ser dois, quando b forma uma ponte);
- os slots que b deixou ficam livres: $livre(s, l_0)$;
- o bloco z ganha espaço em cima nesses slots, $esp(z, s)$, e fica com $clr(z)$ se não sobrou nada em cima dele.

Deixa de valer (DEL):

- a posição antiga de b : $at(b, p_0)$, $lev(b, l_0)$ e $cov(b, s, l_0)$;
- os apoios antigos: $on(b, z)$;
- os slots que b passou a ocupar deixam de estar livres: $livre(s, l^*)$;
- o bloco y' perde o espaço em cima nesses slots, $esp(y', s)$, e perde $clr(y')$.

Se b não mudou de ponto ($p = p_0$) ou não mudou de nível ($l^* = l_0$), esse fato continua igual e não entra em nenhuma das duas listas.

2.4 Elementos novos e as ações associadas

A tabela abaixo mostra o que é novo em relação ao mundo clássico e o que a ação *move* adiciona e remove em cada caso.

Elemento	Para que serve	<i>move</i> adiciona	<i>move</i> remove
$\ell(b)$	tamanho do bloco	não muda	não muda
$at(b, p)$	posição horizontal	$at(b, p)$ de destino	$at(b, p_0)$
$lev(b, l)$	altura	$lev(b, l^*)$	$lev(b, l_0)$
$cov(b, s, l)$	ocupação de slots	slots do novo span	slots do span antigo
$livre(s, l)$	espaços vazios	slots que b deixou	slots que b ocupou
$esp(b, s)$ (novo)	pode receber um bloco no slot s ?	slots do antigo apoio que b deixou	slots do novo apoio onde b pousou
$clr(b)$	pode ser movido?	antigo apoio descoberto	cada novo apoio
$on(b, y)$	apoio (1 ou 2 = ponte)	$on(b, y')$ novos	$on(b, z)$ antigos
estabilidade (A7)	equilíbrio	restrição verificada no estado de chegada	
sombra (P5)	espaço sob bloco suspenso	pré-condição (bloqueia o destino)	

2.5 Ajustes em relação ao mundo clássico

As pré-condições do mundo clássico não são suficientes para blocos de tamanhos diferentes e por causa disso foi necessário realizar uns ajustes.

(i) Um predicado novo no lugar de $clr(y)$. No mundo clássico, para colocar b em cima de y é preciso $clr(y)$. Naquele cenário isso funciona, porque só cabe um bloco em cima de outro. Aqui cabem dois blocos pequenos lado a lado em cima de c . Depois que a vai para cima de c , $clr(c)$ fica falso, e a regra clássica não deixa b ir para o lado de a , mesmo com espaço disponível suficiente. Usando a regra clássica como está:

- os estados S_{f1} , S_{f2} , o S_5 da Situação 2 e o S_7 da Situação 3 não podem ser alcançados. Fizemos o teste colocando $clr(y)$ como pré-condição na nossa CNF: a Situação 1 continua com plano de 4 ações, mas as Situações 2 e 3 ficam sem solução em todos os horizontes que testamos (H de 0 a 12);

- o passo $S_4 \rightarrow S_5$ da figura da Situação 3 seria proibido, e o S_0 da Situação 2, que já começa com a e b lado a lado em cima de c , nunca poderia ser montado.

Por isso criamos o predicado $esp(y, s, t)$, que significa “há espaço livre em cima de y no slot s ” (axioma A5a). Com ele ficamos com duas condições diferentes:

- $clr(b)$ diz que não tem nada em cima do bloco inteiro. É a condição para mover b (P1);
- $esp(y, s)$ diz que não tem nada em cima de y só naquele slot. É a condição para y receber um bloco (P2).

O clr continua existindo com o mesmo significado. Ele só passou a ser escrito usando o esp (A5b).

(ii) Regra da sombra. Com blocos de tamanhos diferentes pode sobrar um espaço vazio embaixo de um bloco. Em S_0 , por exemplo, o slot s_4 fica embaixo da ponte d . As outras pré-condições não impedem que um bloco seja colocado nesse tipo de espaço. Sem uma regra para isso, seria possível ir de S_0 a S_{f3} em 2 ações ($move(d, c, 0)$ e $move(a, T, 2)$), enfiando a embaixo da parte de d que fica para fora de c , o que não dá para fazer com blocos de verdade. Por isso incluímos a pré-condição P5.

Conferência. Com os ajustes (i) e (ii), cada passo desenhado nas figuras das Situações 2 e 3 pode ser feito com uma única ação válida.

2.6 Execução manual

Para cada cenário mostramos o plano desenhado estado por estado. Para cada ação mostramos as pré-condições que conferimos e os efeitos (ADD e DEL). Conferimos os efeitos comparando os fatos verdadeiros antes e depois de cada ação. Para as tabelas não ficarem muito longas, não listamos as mudanças de esp . Elas acompanham as de $livre$: $esp(y, s)$ é apagado nos slots onde b pousa em cima de y e é adicionado nos slots que b deixa no bloco onde estava.

2.6.1 Situação 1: $S_0 \rightarrow S_{f4}$

O plano manual tem 4 ações (Figura 2). A Tabela 1 mostra, para cada passo, as pré-condições verificadas e os efeitos.

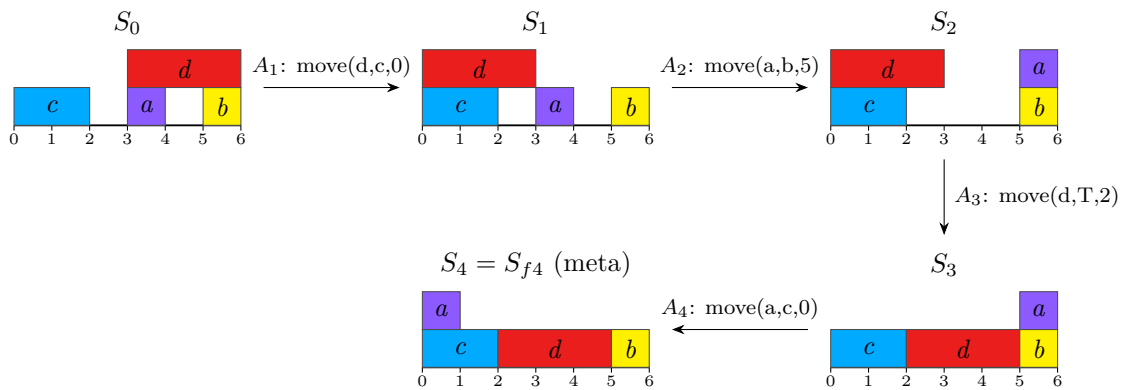


Figura 2: Plano manual de 4 ações.

Ação	Pré-condições (verificadas)	ADD	DEL
A_1 : move(d,c,0) $t = 0$	$clr(d)$; c no nível 0 em $p=0$ e $span(d,0) = \{0,1,2\}$ sobrepõe $span(c,0) = \{0,1\}$; s_0, s_1, s_2 livres no nível 1 e acima; estável: s_0, s_1 ocupados $\Rightarrow 2 \geq \lceil 3/2 \rceil$.	$at(d,0), on(d,c), clr(a),$ $clr(b), livre(s_3..s_5,1)$	$at(d,3), on(d,a), on(d,b),$ $clr(c), livre(s_0..s_2,1)$
A_2 : move(a,b,5) $t = 1$	$clr(a)$ (liberado por A_1); b no nível 0 em $p=5$ e $\{5\}$ sobrepõe $\{5\}$; s_5 livre no nível 1 e acima; estável: s_5 ocupado por b .	$at(a,5), lev(a,1), on(a,b),$ $livre(s_3,0)$	$at(a,3), lev(a,0), on(a,T),$ $clr(b), livre(s_5,1)$
A_3 : move(d,T,2) $t = 2$	$clr(d)$; s_2, s_3, s_4 livres no nível 0 (s_3 liberado por A_2) e acima (a está em s_5 , fora do span).	$at(d,2), lev(d,0), on(d,T),$ $clr(c), livre(s_0..s_2,1)$	$at(d,0), lev(d,1), on(d,c),$ $livre(s_2..s_4,0)$
A_4 : move(a,c,0) $t = 3$	$clr(a)$; c no nível 0 em $p=0$ e $\{0\} \subset \{0,1\}$; s_0 livre no nível 1 e acima (liberado por A_3); estável: s_0 ocupado por c .	$at(a,0), on(a,c), clr(b),$ $livre(s_5,1)$	$at(a,5), on(a,b), clr(c),$ $livre(s_0,1)$

Tabela 1: Situação 1: execução manual passo a passo. Em A_1 e A_4 o nível do bloco não muda (continua 1), então lev não aparece no ADD/DEL.

Por que 4 ações é o mínimo. (1) O primeiro movimento de d não pode ser para a posição final, porque a ocupa s_3 . Ele também não pode ser para a mesa, porque não há 3 slots livres seguidos. Então d vai primeiro para cima de c e se move pelo menos duas vezes. (2) a precisa sair de s_3 antes do segundo movimento de d , mas nesse momento c está coberto por d , então o primeiro movimento de a também não é o final. Logo a também se move pelo menos duas vezes. Somando, são pelo menos 4 ações. O SAT dá o mesmo resultado: com $H = 3$ a fórmula é insatisfatível (Seção 6).

2.6.2 Situação 2: $S_0 \rightarrow S_5$

O plano manual é o da própria figura do enunciado, com 5 ações (Figura 3 e Tabela 2).

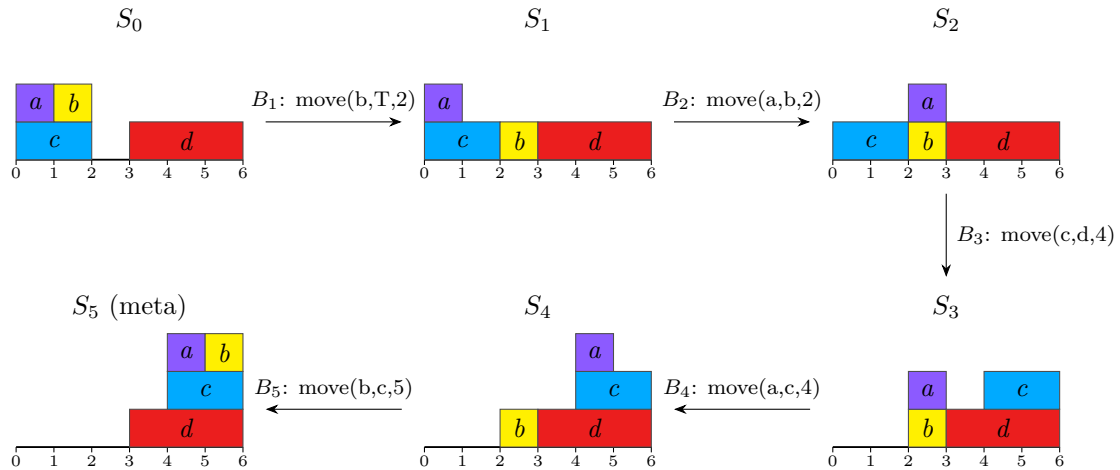


Figura 3: Situação 2: plano manual de 5 ações (o mesmo da figura do enunciado).

Ação	Pré-condições (verificadas)	ADD	DEL
B_1 : move(b,T,2) $t = 0$	$clr(b)$; s_2 livre no nível 0 e acima (sombra); na mesa não há teste de equilíbrio.	$at(b, 2)$, $lev(b, 0)$, $on(b, T)$, $livre(s_1, 1)$	$at(b, 1)$, $lev(b, 1)$, $on(b, c)$, $livre(s_2, 0)$
B_2 : move(a,b,2) $t = 1$	$clr(a)$; b no nível 0 em $p=2$ e $\{2\}$ sobrepõe $\{2\}$; s_2 livre no nível 1 e acima; estável: s_2 apoiado por b .	$at(a, 2)$, $on(a, b)$, $clr(c)$, $livre(s_0, 1)$	$at(a, 0)$, $on(a, c)$, $clr(b)$, $livre(s_2, 1)$
B_3 : move(c,d,4) $t = 2$	$clr(c)$ (b e a saíram em B_1 e B_2); d no nível 0 em $p=3$ e $\{4, 5\} \subset \{3, 4, 5\}$; s_4, s_5 livres no nível 1 e acima; estável: $2 \geq \lceil 2/2 \rceil$.	$at(c, 4)$, $lev(c, 1)$, $on(c, d)$, $livre(s_0, 0)$, $livre(s_1, 0)$	$at(c, 0)$, $lev(c, 0)$, $on(c, T)$, $clr(d)$, $livre(s_4, 1)$, $livre(s_5, 1)$
B_4 : move(a,c,4) $t = 3$	$clr(a)$; c no nível 1 em $p=4$ e $\{4\} \subset \{4, 5\}$; s_4 livre no nível 2 e acima; estável: s_4 apoiado por c .	$at(a, 4)$, $lev(a, 2)$, $on(a, c)$, $clr(b)$, $livre(s_2, 1)$	$at(a, 2)$, $lev(a, 1)$, $on(a, b)$, $clr(c)$, $livre(s_4, 2)$
B_5 : move(b,c,5) $t = 4$	$clr(b)$ (liberado por B_4); c no nível 1 em $p=4$ e $\{5\} \subset \{4, 5\}$; s_5 livre no nível 2 e acima; estável: s_5 apoiado por c . Aqui $clr(c)$ é falso, porque a está em cima de c , mas $esp(c, s_5)$ é verdadeiro. Sem o predicado novo esta ação não seria permitida.	$at(b, 5)$, $lev(b, 2)$, $on(b, c)$, $livre(s_2, 0)$	$at(b, 2)$, $lev(b, 0)$, $on(b, T)$, $livre(s_5, 2)$

Tabela 2: Situação 2: execução manual passo a passo. Em B_2 o nível de a não muda (continua 1).

Por que 5 ações é o mínimo. O bloco c precisa ir para cima de d (pelo menos 1 ação). Mas a e b estão em cima de c e terminam de novo em cima de c , agora no nível 2. Cada um precisa sair de cima de c antes de c se mover e voltar depois, então cada um se move pelo menos duas vezes. Somando, são pelo menos $1 + 2 + 2 = 5$ ações. No SAT, com $H = 4$ a fórmula é insatisfatível. Existe mais de um plano com 5 ações, e o SAT devolveu um que não é o da figura (Seção 7).

2.6.3 Situação 3: $S_0 \rightarrow S_7$

O plano manual tem 6 ações (Figura 4 e Tabela 3). O S_0 é o mesmo da Situação 1, e os quatro primeiros passos são os mesmos do plano de lá. Depois deles o estado é $S_4 = S_{f4}$. Na figura do enunciado os estados S_5 e S_6 são iguais, então tratamos os dois como um estado só.

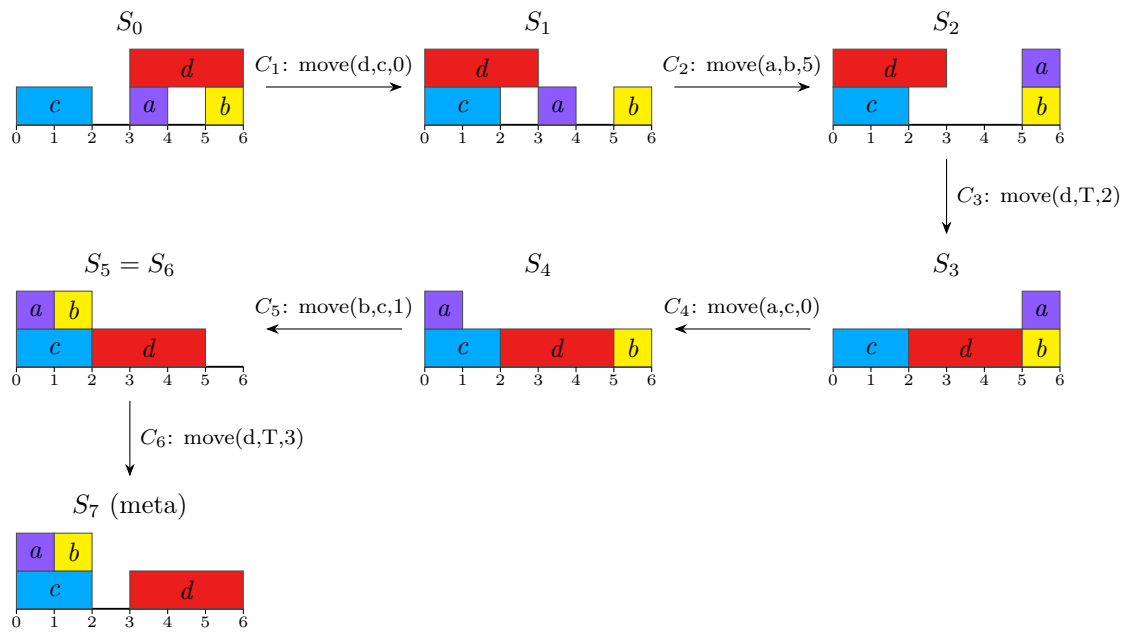


Figura 4: Situação 3: plano manual de 6 ações.

Ação	Pré-condições (verificadas)	ADD	DEL
C_1 : move(d,c,0) $t = 0$	$clr(d)$; c no nível 0 em $p=0$ e $span(d, 0) = \{0, 1, 2\}$ sobrepõe $span(c, 0) = \{0, 1\}$; s_0, s_1, s_2 livres no nível 1 e acima; estável: s_0, s_1 ocupados $\Rightarrow 2 \geq \lceil 3/2 \rceil$.	$at(d, 0), on(d, c), clr(a),$ $clr(b), livre(s_3..s_5, 1)$	$at(d, 3), on(d, a), on(d, b),$ $clr(c), livre(s_0..s_2, 1)$
C_2 : move(a,b,5) $t = 1$	$clr(a)$ (liberado por C_1); b no nível 0 em $p=5$ e $\{5\}$ sobrepõe $\{5\}$; s_5 livre no nível 1 e acima; estável: s_5 ocupado por b .	$at(a, 5), lev(a, 1), on(a, b),$ $livre(s_3, 0)$	$at(a, 3), lev(a, 0), on(a, T),$ $clr(b), livre(s_5, 1)$
C_3 : move(d,T,2) $t = 2$	$clr(d)$; s_2, s_3, s_4 livres no nível 0 (s_3 liberado por C_2) e acima (a está em s_5 , fora do span).	$at(d, 2), lev(d, 0), on(d, T),$ $clr(c), livre(s_0..s_2, 1)$	$at(d, 0), lev(d, 1), on(d, c),$ $livre(s_2..s_4, 0)$
C_4 : move(a,c,0) $t = 3$	$clr(a)$; c no nível 0 em $p=0$ e $\{0\} \subset \{0, 1\}$; s_0 livre no nível 1 e acima (liberado por C_3); estável: s_0 ocupado por c .	$at(a, 0), on(a, c), clr(b),$ $livre(s_5, 1)$	$at(a, 5), on(a, b), clr(c),$ $livre(s_0, 1)$
C_5 : move(b,c,1) $t = 4$	$clr(b)$ (a saiu de cima de b em C_4); c no nível 0 em $p=0$ e $\{1\} \subset \{0, 1\}$; s_1 livre no nível 1 e acima (a ocupa só s_0); estável: s_1 apoiado por c . Aqui $clr(c)$ é falso, porque a está em cima de c , mas $esp(c, s_1)$ é verdadeiro. Sem o predicado novo esta ação não seria permitida.	$at(b, 1), lev(b, 1), on(b, c),$ $livre(s_5, 0)$	$at(b, 5), lev(b, 0), on(b, T),$ $livre(s_1, 1)$
C_6 : move(d,T,3) $t = 5$	$clr(d)$; s_3, s_4, s_5 livres no nível 0 sem contar o próprio d (s_5 liberado em C_5) e acima (s_5 no nível 1 está livre desde C_4). É um movimento lateral, em que d anda uma posição na mesa.	$at(d, 3), livre(s_2, 0)$	$at(d, 2), livre(s_5, 0)$

Tabela 3: Situação 3: execução manual passo a passo. Em C_1 e C_4 o nível do bloco não muda (continua 1). Em C_6 o nível e os apoios de d não mudam.

Por que 6 ações é o mínimo. (1) O primeiro movimento de d não pode ser para o lugar final, porque a ocupa s_3 e b ocupa s_5 . Ele também não pode ser para outro lugar da mesa, porque não há 3 slots livres seguidos. Então d vai primeiro para cima de c e se move pelo menos duas vezes. (2) a precisa sair de s_3 antes de d descer para a mesa, mas nesse momento c está coberto por d , então o primeiro movimento de a não é o final. Logo a também se move pelo menos duas vezes. (3) b tem que sair de s_5 e ir para cima de c , e d só pode ir para $p = 3$ depois que s_5 ficar livre. Se b se move uma única vez (direto para cima de c), isso só acontece depois de d sair de cima de c para a mesa em $p = 2$, e então d precisa de um terceiro movimento. Se d se move só duas vezes, b precisa de um movimento a mais para sair do caminho. Nos dois casos são pelo menos 6 ações. No SAT, com $H = 5$ a fórmula é insatisfatível.

2.7 Ordem parcial

Um plano de ordem parcial não fixa a ordem de todas as ações. Ele só fixa a ordem que é realmente necessária. Ele é formado por:

- ações, incluindo duas que não movem nada: *Início*, que tem como efeitos os fatos de S_0 , e *Fim*, que tem como pré-condições os fatos da meta;
- vínculos causais $A \xrightarrow{p} B$, que significam que a ação A produz a condição p e que p é pré-condição de B ;
- restrições de ordem $A \prec B$ (“ A antes de B ”), que vêm dos vínculos causais.

Uma ação C é uma ameaça ao vínculo $A \xrightarrow{p} B$ se ela apaga p e pode acontecer entre A e B . Para resolver a ameaça, colocamos C depois de B (promoção, $B \prec C$) ou antes de A (rebaixamento, $C \prec A$). Uma linearização é uma ordem completa das ações que respeita todas as restrições.

2.7.1 Situação 1

Vínculo causal	Condição p	Ordem gerada
Início $\rightarrow A_1$	$clr(d)$, c em $(0, 0)$, $s_0..s_2$ livres no nível 1	nenhuma
$A_1 \rightarrow A_2$	$clr(a)$, $livre(s_5, 1)$	$A_1 \prec A_2$
Início $\rightarrow A_2$	b em $(5, 0)$ (apoio)	nenhuma
$A_2 \rightarrow A_3$	$livre(s_3, 0)$	$A_2 \prec A_3$
$A_1 \rightarrow A_3$	$livre(s_3, 1)$, $livre(s_4, 1)$: nada acima do destino (sombra)	$A_1 \prec A_3$
Início $\rightarrow A_3$	$clr(d)$, $livre(s_2, 0)$, $livre(s_4, 0)$	nenhuma
$A_1 \rightarrow A_4$	$clr(a)$	$A_1 \prec A_4$
$A_3 \rightarrow A_4$	$livre(s_0, 1)$ (c descoberto)	$A_3 \prec A_4$
$A_3 \rightarrow \text{Fim}$	$at(d, 2) \wedge lev(d, 0)$	nenhuma
$A_4 \rightarrow \text{Fim}$	$at(a, 0) \wedge lev(a, 1)$	nenhuma
Início $\rightarrow \text{Fim}$	$at(b, 5)$, $at(c, 0)$, $lev(b, 0)$, $lev(c, 0)$	nenhuma

Tabela 4: Situação 1: vínculos causais do plano $S_0 \rightarrow S_{f_4}$.

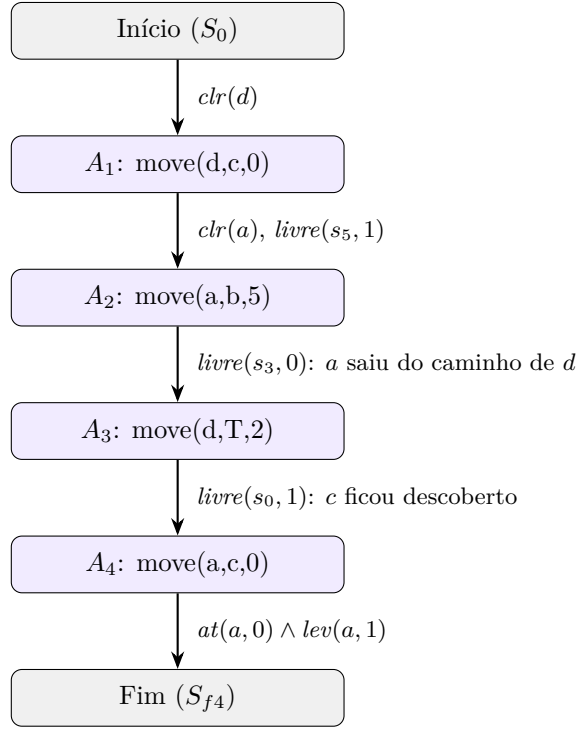


Figura 5: Grafo de ordem parcial da Situação 1. As setas são os vínculos causais que fixam a ordem. Os vínculos entre ações que não são vizinhas estão na Tabela 4.

Análise de ameaças.

- A_2 apaga $clr(b)$ e $livre(s_5, 1)$: nenhuma ação posterior precisa delas. Sem ameaça.
- A_3 apaga $livre(s_2..s_4, 0)$: só a Fim usa esses slots (com d). Sem ameaça.
- O vínculo $A_1 \xrightarrow{clr(a)} A_4$ só seria ameaçado por uma ação que pusesse um bloco sobre a entre A_1 e A_4 , e nenhuma ação faz isso.

Um exemplo de ameaça sem solução. Se trocarmos A_2 por $move(a, T, 4)$, que parece mais simples porque põe a direto na mesa, essa ação apaga $livre(s_4, 0)$, que A_3 precisa (vínculo $Início \xrightarrow{livre(s_4, 0)} A_3$). Rebaixamento (antes do Início) é impossível. Promoção ($A_3 \prec move(a, T, 4)$) também é, porque A_3 depende de a ter saído de s_3 . A ameaça não tem solução, e esse plano é inválido.

Conclusão. As restrições $A_1 \prec A_2 \prec A_3 \prec A_4$ formam uma cadeia, então só existe uma linearização. Isso acontece porque cada ação libera o caminho da próxima: d está em cima de a , a está no caminho de d , e d está em cima do lugar final de a . Em outros cenários há mais liberdade. Na Situação 2, por exemplo, existe um plano ótimo com 4 linearizações.

2.7.2 Situação 2

Vínculo causal	Condição p	Ordem gerada
Início $\rightarrow B_1$	$clr(b)$, $livre(s_2, 0)$	nenhuma
$B_1 \rightarrow B_2$	b em $(2, 0)$ (apoio de a)	$B_1 \prec B_2$
Início $\rightarrow B_2$	$clr(a)$, $livre(s_2, 1)$	nenhuma
$B_1 \rightarrow B_3$	$livre(s_1, 1)$: b saiu de cima de c	$B_1 \prec B_3$
$B_2 \rightarrow B_3$	$clr(c)$: a saiu de cima de c	$B_2 \prec B_3$
Início $\rightarrow B_3$	d em $(3, 0)$, $livre(s_4, 1)$, $livre(s_5, 1)$	nenhuma
$B_3 \rightarrow B_4$	c em $(4, 1)$	$B_3 \prec B_4$
$B_3 \rightarrow B_5$	c em $(4, 1)$	$B_3 \prec B_5$
$B_4 \rightarrow B_5$	$clr(b)$: a saiu de cima de b	$B_4 \prec B_5$
$B_3, B_4, B_5 \rightarrow \text{Fim}$	c, a, b nos lugares finais	nenhuma
Início $\rightarrow \text{Fim}$	d em $(3, 0)$	nenhuma

Tabela 5: Situação 2: vínculos causais do plano da figura.

Ameaça. B_2 coloca a em cima de b e apaga $clr(b)$, que B_5 precisa. A única ação que devolve $clr(b)$ é B_4 , quando a sai de cima de b . Por isso é preciso $B_4 \prec B_5$. Com isso o plano da figura vira uma cadeia $B_1 \prec B_2 \prec B_3 \prec B_4 \prec B_5$, com uma única linearização.

Um plano alternativo com mais de uma ordem possível. A ameaça existe porque o a foi guardado em cima do b . Se guardarmos o a em cima do d , temos outro plano ótimo de 5 ações, sem essa dependência (Figura 6): $\{\text{move}(b, T, 2), \text{move}(a, d, 3)\} \prec \text{move}(c, d, 4) \prec \{\text{move}(a, c, 4), \text{move}(b, c, 5)\}$. As duas ações de cada par não dependem uma da outra, porque os blocos já estão livres e vão para slots diferentes. Então cada par pode ser executado em qualquer ordem, e esse plano parcial tem $2 \times 2 = 4$ linearizações.

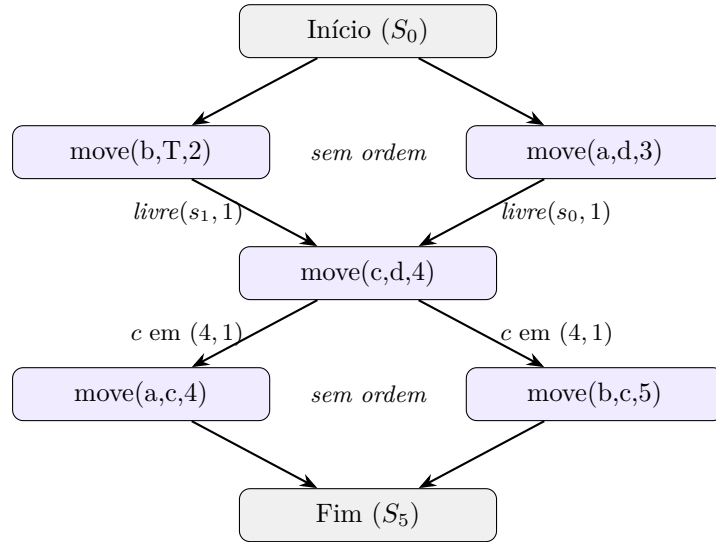


Figura 6: Situação 2: plano alternativo de ordem parcial, com 4 linearizações.

2.7.3 Situação 3

O plano tem seis ações: $C_1 = \text{move}(d, c, 0)$, $C_2 = \text{move}(a, b, 5)$, $C_3 = \text{move}(d, T, 2)$, $C_4 = \text{move}(a, c, 0)$, $C_5 = \text{move}(b, c, 1)$ e $C_6 = \text{move}(d, T, 3)$. A Tabela 6 mostra, para cada pré-condição de cada ação, qual ação produz essa condição.

Vínculo causal	Condição p	Ordem gerada
Início $\rightarrow C_1$	$clr(d)$, c em $(0,0)$, $s_0..s_2$ livres no nível 1	nenhuma
$C_1 \rightarrow C_2$	$clr(a)$, $livre(s_5, 1)$: d saiu de cima de a e de b	$C_1 \prec C_2$
Início $\rightarrow C_2$	b em $(5,0)$ (apoio de a)	nenhuma
$C_2 \rightarrow C_3$	$livre(s_3, 0)$: a saiu do caminho de d	$C_2 \prec C_3$
$C_1 \rightarrow C_3$	$livre(s_3, 1)$, $livre(s_4, 1)$: nada acima do destino (sombra)	$C_1 \prec C_3$
Início $\rightarrow C_3$	$clr(d)$, $livre(s_2, 0)$, $livre(s_4, 0)$	nenhuma
$C_1 \rightarrow C_4$	$clr(a)$	$C_1 \prec C_4$
$C_3 \rightarrow C_4$	$livre(s_0, 1)$: d saiu de cima de c	$C_3 \prec C_4$
Início $\rightarrow C_4$	c em $(0,0)$ (apoio de a)	nenhuma
$C_4 \rightarrow C_5$	$clr(b)$: a saiu de cima de b	$C_4 \prec C_5$
$C_3 \rightarrow C_5$	$livre(s_1, 1)$: d saiu de cima de c	$C_3 \prec C_5$
Início $\rightarrow C_5$	c em $(0,0)$ (apoio de b)	nenhuma
$C_5 \rightarrow C_6$	$livre(s_5, 0)$: b saiu da mesa	$C_5 \prec C_6$
$C_4 \rightarrow C_6$	$livre(s_5, 1)$: a saiu de cima de s_5 (sombra)	$C_4 \prec C_6$
$C_1 \rightarrow C_6$	$livre(s_3, 1)$, $livre(s_4, 1)$: nada acima do destino (sombra)	$C_1 \prec C_6$
Início $\rightarrow C_6$	$clr(d)$	nenhuma
$C_4 \rightarrow \text{Fim}$	$at(a, 0) \wedge lev(a, 1)$	nenhuma
$C_5 \rightarrow \text{Fim}$	$at(b, 1) \wedge lev(b, 1)$	nenhuma
$C_6 \rightarrow \text{Fim}$	$at(d, 3) \wedge lev(d, 0)$	nenhuma
Início $\rightarrow \text{Fim}$	$at(c, 0) \wedge lev(c, 0)$	nenhuma

Tabela 6: Situação 3: vínculos causais do plano $S_0 \rightarrow S_7$, agrupados pela ação que usa a condição.

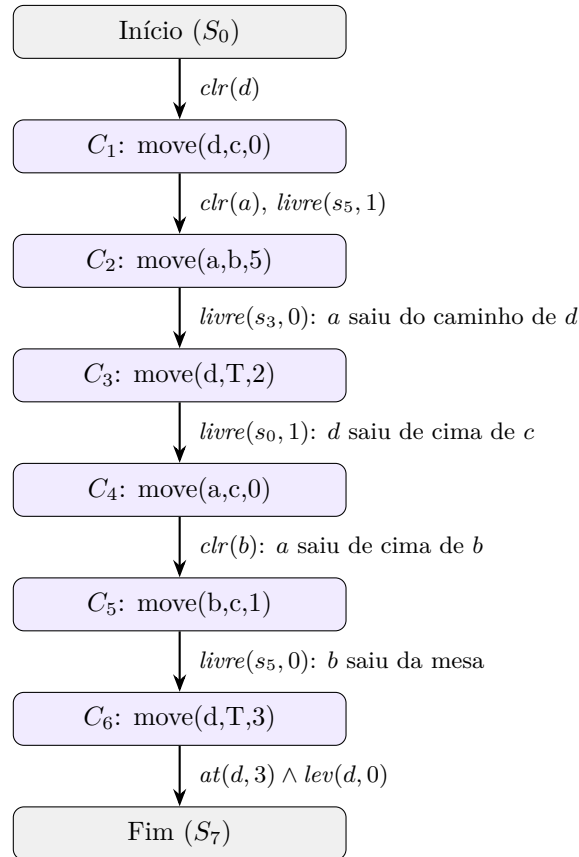


Figura 7: Grafo de ordem parcial da Situação 3. As setas são os vínculos causais que fixam a ordem. Os vínculos entre ações que não são vizinhas estão na Tabela 6.

Análise de ameaças. Para cada ação, olhamos o que ela apaga e se alguma outra ação precisa dessa condição.

- C_1 apaga $clr(c)$ e $livre(s_0..s_2, 1)$. C_4 e C_5 precisam de $livre(s_0, 1)$ e $livre(s_1, 1)$, mas quem produz essas condições é C_3 , que já vem depois de C_1 . Sem ameaça.
- C_2 apaga $clr(b)$ e $livre(s_5, 1)$. C_5 precisa de $clr(b)$ e C_6 precisa de $livre(s_5, 1)$, mas as duas condições são devolvidas por C_4 , que já vem depois de C_2 ($C_2 \prec C_3 \prec C_4$). Sem ameaça.
- C_3 apaga $livre(s_2..s_4, 0)$. Depois dela só o próprio d usa esses slots, em C_6 . Sem ameaça.
- C_4 apaga $clr(c)$ e $livre(s_0, 1)$. C_5 também põe um bloco em cima de c , mas só precisa de espaço no slot s_1 ($livre(s_1, 1)$), que C_4 não apaga. Sem ameaça.
- C_5 tira b de $(5, 0)$, que é o apoio de a em C_2 (vínculo Início $\rightarrow C_2$). Como C_5 precisa de $clr(b)$, que só volta em C_4 , ela já fica depois de C_2 . C_5 também apaga $livre(s_1, 1)$, que nenhuma ação usa depois. Sem ameaça.
- C_6 apaga $livre(s_5, 0)$, que nenhuma ação usa depois. Sem ameaça.
- Os vínculos de $clr(d)$ e de c em $(0, 0)$ não são ameaçados por nenhuma ação, porque nenhuma ação põe um bloco em cima de d e o bloco c não se move.

Conclusão. As restrições $C_1 \prec C_2$, $C_2 \prec C_3$, $C_3 \prec C_4$, $C_4 \prec C_5$ e $C_5 \prec C_6$ formam uma cadeia, então só existe uma linearização: $C_1, C_2, C_3, C_4, C_5, C_6$. Cada ação libera o que a próxima precisa. C_1 descobre a . C_2 tira a do caminho de d . C_3 descobre c . C_4 descobre b e libera o espaço acima de s_5 . C_5 libera s_5 na mesa. C_6 coloca d no lugar final.

2.7.4 Ordem parcial no SAT

Uma restrição de ordem parcial $\varphi_1 \prec_P \varphi_2$ (“ φ_1 acontece antes de φ_2 ”) vira as cláusulas $\neg\varphi_2(t) \vee \bigvee_{t' \leq t} \varphi_1(t')$ para todo t . Esse é o grupo 3.7 do código. Escolhemos uma restrição por cenário, tirada de um vínculo causal dos planos acima:

Cenário	Vínculo	$\varphi_1 \prec_P \varphi_2$	Resultado
Sit. 1	$A_3 \rightarrow A_4$	d em $(2, 0) \prec_P a$ em $(0, 1)$	SAT com $H = 4$; invertida: UNSAT para $H = 3..8$
Sit. 2	$B_3 \rightarrow B_4$	c em $(4, 1) \prec_P a$ em $(4, 2)$	SAT com $H = 5$; invertida: mínimo passa a 7
Sit. 3	$C_5 \rightarrow C_6$	b em $(1, 1) \prec_P d$ em $(3, 0)$	SAT com $H = 6$; invertida: mínimo passa a 7

Com a ordem certa, o plano ótimo continua existindo. Com a ordem invertida ele deixa de existir: na Situação 1 não achamos nenhum plano (testamos H até 8), e nas Situações 2 e 3 o menor plano passa a ter 7 ações. Isso mostra que as ordens que achamos na análise manual são mesmo obrigatórias.

3 Codificação CNF

Para passar da LPO para a lógica proposicional, criamos uma variável verdadeiro/falso para cada predicado e cada combinação possível de valores (bloco, posição, nível e instante). As regras viram cláusulas, que são um “ou” de variáveis ou de negações de variáveis.

3.1 Variáveis proposicionais

Variável	Índices	Tipo	Sit. 1 $H = 4$	Sit. 2 $H = 5$	Sit. 3 $H = 6$
$at(b, p, t)$	$p \in [0, 6 - \ell(b)], t \in [0, H]$	básico	105	126	147
$lev(b, l, t)$	$l \in [0, 3], t \in [0, H]$	básico	80	96	112
$clr(b, t)$	$t \in [0, H]$	calculado	20	24	28
$cov(b, s, l, t)$	$s \in [0, 5], l \in [0, 3], t \in [0, H]$	auxiliar	480	576	672
$move(b, y, p, t)$	$y \in \mathcal{B} \cup \{\mathcal{T}\}, y \neq b, t \in [0, H - 1]$	ação	336	420	504
$\varphi(\cdot, t)$	$t \in [0, H]$	aux. (ordem parcial)	5	6	7
$esp(b, s, t)$	$s \in [0, 5], t \in [0, H]$	calculado (novo)	120	144	168
Total			1146	1392	1638

Usamos a variável auxiliar cov para as cláusulas ficarem mais curtas. Com ela, as regras de exclusão, de estabilidade e de *clear* são escritas como “o slot está ocupado”, sem precisar combinar at e lev toda vez.

3.2 Grupos de cláusulas

Grupo	Formato das cláusulas	Qtde
3.1 Estado inicial	unitárias $at(b, p, 0), lev(b, l, 0)$	8
3.2 Meta	unitárias $at(b, p, H), lev(b, l, H)$	8
3.3 (A) Posição	$\bigvee_p at(b, p, t); \neg at(b, p_1, t) \vee \neg at(b, p_2, t)$	250
3.3 (B) Nível	$\bigvee_l lev(b, l, t); \neg lev(b, l_1, t) \vee \neg lev(b, l_2, t)$	140
3.3 (Z) cov	$\neg cov \vee lev(b, l, t); \neg cov \vee \bigvee_{p \ni s} at(b, p, t); \neg at(b, p, t) \vee \neg lev(b, l, t) \vee cov(b, s, l, t)$	1640
3.3 (C) Exclusão	$\neg cov(b_1, s, l, t) \vee \neg cov(b_2, s, l, t)$	720
3.3 (D) Estabilidade	para todo $S \subseteq span(b, p)$ com $ S = \ell(b) - \lceil \ell(b)/2 \rceil + 1$: $\neg at(b, p, t) \vee \neg lev(b, l, t) \vee \bigvee_{s \in S, b' \neq b} cov(b', s, l - 1, t)$	435
3.3 (Y) esp (novo)	$\neg esp(b, s, t) \vee \bigvee_l cov(b, s, l, t); \neg esp \vee \neg cov(b, s, l, t) \vee \neg cov(b', s, l + 1, t); \neg cov(b, s, l, t) \vee esp \vee \bigvee_{b'} cov(b', s, l + 1, t)$	1680
3.3 (E) Clear	$\neg clr(b, t) \vee \neg cov(b, s, l, t) \vee esp(b, s, t); \neg at(b, p, t) \vee clr(b, t) \vee \bigvee_{s \in span(b, p)} \neg esp(b, s, t)$	585
3.4 (G) Pré-condições	G1 $\neg move \vee clr(b, t)$; G2 $\neg move \vee \neg at(y, q, t) \vee \neg lev(y, l, t) \vee esp(y, s, t) \vee cov(b, s, l + 1, t)$; G3 destino diferente do lugar atual; G4 $\neg move \vee \bigvee_q at(y, q, t)$ (spans sobrepostos); G5+G6 $\neg move [\vee \neg lev(y, l, t)] \vee \neg cov(b', s, l', t)$ para $l' \geq l^*$; G7 $\neg move \vee \neg lev(y, 3, t)$	12384
3.4 (F) Efeitos	F1 $\neg move \vee at(b, p, t + 1)$; F2 $\neg move \vee \neg lev(y, l, t) \vee lev(b, l + 1, t + 1)$; F3 $\neg move \vee \neg clr(y, t + 1)$	1428
3.5 Frame axioms	$at(b, p, t) \vee \neg at(b, p, t + 1) \vee \bigvee move(b, \cdot, \cdot, t)$ e o simétrico; idem para lev	296
3.6 Ação única	$\bigvee move(\cdot, \cdot, \cdot, t); \neg move_i \vee \neg move_j$	13948
3.7 Ordem parcial	$\neg \varphi_2(t) \vee \bigvee_{t' \leq t} \varphi_1(t')$ (+ definição de φ_1)	20
Total		33542

Observações sobre a tabela:

- Na estabilidade precisamos dizer “pelo menos k de n slots estão apoiados”. Em cláusulas isso é escrito como “em todo grupo de $n - k + 1$ slots, pelo menos um está apoiado”. Para d ($n = 3, k = 2$) isso dá 3 cláusulas para cada posição e nível.
- O efeito “ b deixa a posição antiga” (F4) não precisa de cláusula própria, porque a unicidade (A) já garante isso.

- *clr* e *esp* não precisam de *frame axiom*. Os grupos (Y) e (E) calculam os dois de novo em cada instante, a partir de *cov*.
- Em G2, o literal $cov(b, s, l+1, t)$ serve para não contar o próprio *b*. Num movimento lateral em cima do mesmo apoio, o slot que o próprio *b* ocupa conta como livre.

Quantidade de cláusulas por cenário. A tabela acima mostra a Situação 1. Nos três cenários os tipos de cláusula são os mesmos. O que muda é o horizonte *H*:

Grupo	Sit. 1 ($H = 4$)	Sit. 2 ($H = 5$)	Sit. 3 ($H = 6$)
3.1 Estado inicial / 3.2 Meta	8 / 8	8 / 8	8 / 8
3.3 (A) Posição / (B) Nível	250 / 140	300 / 168	350 / 196
3.3 (Z) <i>cov</i>	1640	1968	2296
3.3 (C) Exclusão	720	864	1008
3.3 (D) Estabilidade	435	522	609
3.3 (Y) <i>esp</i>	1680	2016	2352
3.3 (E) Clear	585	702	819
3.4 (G) Pré-condições	12384	15480	18576
3.4 (F) Efeitos	1428	1785	2142
3.5 Frame axioms	296	370	444
3.6 Ação única	13948	17435	20922
3.7 Ordem parcial	20	24	28
Total	33542	41650	49758

3.3 A relação *on* não vira variável

A relação $on(b, y, t)$ não é uma variável da CNF. O `interpretar.py` calcula o *on* depois, a partir de *at*, *lev* e da sobreposição dos spans (axioma A6).

4 Exemplos dos 3 cenários codificados para CNF em LP

4.1 Situação 1: S_0 e meta S_{f4}

S_0 ($t = 0$)		S_{f4} ($t = H = 4$)	
Fórmula	DIMACS	Fórmula	DIMACS
$at(c, 0, 0) \wedge lev(c, 0, 0)$	13 0, 114 0	$at(c, 0, 4) \wedge lev(c, 0, 4)$	97 0, 178 0
$at(a, 3, 0) \wedge lev(a, 0, 0)$	4 0, 106 0	$at(a, 0, 4) \wedge lev(a, 1, 4)$	85 0, 171 0
$at(b, 5, 0) \wedge lev(b, 0, 0)$	12 0, 110 0	$at(d, 2, 4) \wedge lev(d, 0, 4)$	104 0, 182 0
$at(d, 3, 0) \wedge lev(d, 1, 0)$	21 0, 119 0	$at(b, 5, 4) \wedge lev(b, 0, 4)$	96 0, 174 0

Relações *on* em S_0 : $on(c, T)$, $on(a, T)$, $on(b, T)$, $on(d, a) \wedge on(d, b)$ (ponte). Em S_{f4} : $on(c, T)$, $on(d, T)$, $on(b, T)$, $on(a, c)$.

4.2 Situação 1: exemplos de cláusulas geradas

Algumas cláusulas reais do arquivo `trab01_blocos2SAT.cnf`, com a tradução pelo `.map` (id 762 = $move(d, c, 0, 0)$, a primeira ação do plano):

Grupo	Cláusula	DIMACS
G1	$\neg move(d, c, 0, 0) \vee clr(d, 0)$	-762 189 0
G4	$\neg move(d, c, 0, 0) \vee at(c, 0, 0) \vee at(c, 1, 0) \vee at(c, 2, 0)$	-762 13 14 15 0
G5	$\neg move(d, c, 0, 0) \vee \neg lev(c, 0, 0) \vee \neg cov(a, s_2, 1, 0)$	-762 -114 -214 0
F1	$\neg move(d, c, 0, 0) \vee at(d, 0, 1)$	-762 39 0
F2	$\neg move(d, c, 0, 0) \vee \neg lev(c, 0, 0) \vee lev(d, 1, 1)$	-762 -114 135 0
F3	$\neg move(d, c, 0, 0) \vee \neg clr(c, 1)$	-762 -192 0
(C)	$\neg cov(a, s_3, 0, 0) \vee \neg cov(d, s_3, 0, 0)$	-209 -281 0
(D)	$\neg at(d, 0, 1) \vee \neg lev(d, 1, 1) \vee cov(a, s_0, 0, 1) \vee cov(b, s_0, 0, 1) \vee cov(c, s_0, 0, 1) \vee cov(a, s_1, 0, 1) \vee cov(b, s_1, 0, 1) \vee cov(c, s_1, 0, 1)$	-39 -135 302 326 350 303 327 351 0
G2	$\neg move(d, c, 0, 0) \vee \neg at(c, 0, 0) \vee \neg lev(c, 0, 0) \vee esp(c, s_0, 0) \vee cov(d, s_0, 1, 0)$	-762 -13 -114 1039 284 0
(Y)	$\neg esp(a, s_3, 0) \vee \neg cov(a, s_3, 0, 0) \vee \neg cov(d, s_3, 1, 0)$	-1030 -209 -287 0
(Y)	$\neg cov(a, s_3, 0, 0) \vee esp(a, s_3, 0) \vee cov(b, s_3, 1, 0) \vee cov(c, s_3, 1, 0) \vee cov(d, s_3, 1, 0)$	-209 1030 239 263 287 0
(E)	$\neg clr(a, 0) \vee \neg cov(a, s_3, 0, 0) \vee esp(a, s_3, 0)$	-186 -209 1030 0
(E)	$\neg at(a, 3, 0) \vee clr(a, 0) \vee \neg esp(a, s_3, 0)$	-4 186 -1030 0
3.7	$\neg at(a, 0, 2) \vee \neg lev(a, 1, 2) \vee \varphi(d, 2, 0, 0) \vee \varphi(d, 2, 0, 1) \vee \varphi(d, 2, 0, 2)$	-43 -139 1022 1023 1024 0

As duas cláusulas (Y) definem $esp(a, s_3, 0)$. Em S_0 o bloco d cobre s_3 no nível 1, então $esp(a, s_3, 0)$ é falso. Pela primeira cláusula (E), $clr(a, 0)$ também é falso. A cláusula (D) do exemplo diz que, se d está no nível 1 começando no ponto 0, então pelo menos um dos slots s_0 e s_1 do nível 0 está ocupado. Existem mais duas cláusulas iguais a essa, para os pares $\{s_0, s_2\}$ e $\{s_1, s_2\}$. As três juntas garantem pelo menos 2 slots apoiados.

4.3 Situação 2: S_0 e meta S_5

S_0 ($t = 0$)		S_5 ($t = H = 5$)	
Fórmula	DIMACS	Fórmula	DIMACS
$at(c, 0, 0) \wedge lev(c, 0, 0)$	13 0, 135 0	$at(d, 3, 5) \wedge lev(d, 0, 5)$	126 0, 219 0
$at(d, 3, 0) \wedge lev(d, 0, 0)$	21 0, 139 0	$at(c, 4, 5) \wedge lev(c, 1, 5)$	122 0, 216 0
$at(a, 0, 0) \wedge lev(a, 1, 0)$	1 0, 128 0	$at(a, 4, 5) \wedge lev(a, 2, 5)$	110 0, 209 0
$at(b, 1, 0) \wedge lev(b, 1, 0)$	8 0, 132 0	$at(b, 5, 5) \wedge lev(b, 2, 5)$	117 0, 213 0

Relações *on* em S_0 : $on(c, T)$, $on(d, T)$, $on(a, c)$, $on(b, c)$ (dois blocos lado a lado sobre c). Em S_5 : $on(d, T)$, $on(c, d)$, $on(a, c)$, $on(b, c)$.

4.4 Situação 3: S_0 e meta S_7

S_0 ($t = 0$)		S_7 ($t = H = 6$)	
Fórmula	DIMACS	Fórmula	DIMACS
$at(c, 0, 0) \wedge lev(c, 0, 0)$	13 0, 156 0	$at(c, 0, 6) \wedge lev(c, 0, 6)$	139 0, 252 0
$at(a, 3, 0) \wedge lev(a, 0, 0)$	4 0, 148 0	$at(a, 0, 6) \wedge lev(a, 1, 6)$	127 0, 245 0
$at(b, 5, 0) \wedge lev(b, 0, 0)$	12 0, 152 0	$at(b, 1, 6) \wedge lev(b, 1, 6)$	134 0, 249 0
$at(d, 3, 0) \wedge lev(d, 1, 0)$	21 0, 161 0	$at(d, 3, 6) \wedge lev(d, 0, 6)$	147 0, 256 0

Os ids mudam de um cenário para outro porque o número de instantes muda. Por isso cada cenário tem o seu próprio `.map`.

O predicado novo nas cláusulas. A ação $C_5 = move(b, c, 1, 4)$ (id 1327) coloca b ao lado de a , em cima de c . As cláusulas geradas para ela incluem:

Grupo	Cláusula	DIMACS
G1	$\neg move(b, c, 1, 4) \vee clr(b, 4)$	-1327 277 0
G2	$\neg move(b, c, 1, 4) \vee \neg at(c, 0, 4) \vee \neg lev(c, 0, 4) \vee esp(c, s_1, 4) \vee cov(b, s_1, 1, 4)$	-1327 -97 -220 1580 703 0
G4	$\neg move(b, c, 1, 4) \vee at(c, 0, 4) \vee at(c, 1, 4)$	-1327 97 98 0
G5	$\neg move(b, c, 1, 4) \vee \neg lev(c, 0, 4) \vee \neg cov(a, s_1, 1, 4)$	-1327 -220 -679 0
F1	$\neg move(b, c, 1, 4) \vee at(b, 1, 5)$	-1327 113 0
F3	$\neg move(b, c, 1, 4) \vee \neg clr(c, 5)$	-1327 -282 0

A cláusula G2 exige $esp(c, s_1, 4)$ (id 1580), ou seja, espaço livre em cima de c só no slot s_1 , que é onde b vai pousar. No instante 4, $clr(c, 4)$ (id 278) é falso, porque a está em cima de c no slot s_0 . Já $esp(c, s_1, 4)$ é verdadeiro. Com a pré-condição clássica $clr(y)$ a cláusula seria $\neg move(b, c, 1, 4) \vee clr(c, 4)$, e a Situação 3 daria UNSAT.

5 Mapeamento: Descrição Formal $\rightarrow$ Código

Regra formal	Código (bw2cnf_var.py)
Blocos e comprimentos	BLOCKS = {'a':1, 'b':1, 'c':2, 'd':3}
Pontos / slots / níveis	MAX_POINT = 6, SLOTS, MAX_LEVEL = 3
Posições válidas	valid_positions(L) = range(MAX_POINT - L + 1)
$span(b, p)$, $overlap$	span(b, p), spans_overlap(b1, p1, b2, p2)
at , lev , clr , cov , esp , $move$	dicionários at, lev, clr, cov, esp, mv com f.new_var(nome)
Cenários (S_0 , meta, H)	CENARIOS[N] com INITIAL, GOAL, HORIZON; --cenario N
Estado inicial / meta	blocos # 3.1 ESTADO INICIAL, # 3.2 META
A1 unicidade	# 3.3 (A), # 3.3 (B)
A2 definição de cov	# 3.3 (Z)
A3 exclusão	# 3.3 (C)
A7 estabilidade	# 3.3 (D)
A5a esp (predicado novo)	dicionário esp, # 3.3 (Y)
A5b clear	# 3.3 (E)
Pré-condições P1–P6	# 3.4 (G) (G1, G2, G3, G4, G5+G6, G7)
Efeitos (ADD/DEL)	# 3.4 (F) (F1–F3; F4 pela unicidade)
Frame axioms	# 3.5
Ação única por passo	# 3.6
Ordem parcial $\varphi_1 \prec_P \varphi_2$	ORDEM_PARCIAL, # 3.7
A6 relação on (calculada depois)	interpretar.py: derivar_on

6 Execução Passo a Passo

1. Os cenários ficam no dicionário CENARIOS do bw2cnf_var.py e são escolhidos com --cenario N:

```
CENARIOS = {
  1: dict(INITIAL={'c': (0,0), 'a': (3,0), 'b': (5,0), 'd': (3,1)}, # S0
          GOAL   ={'c': (0,0), 'a': (0,1), 'd': (2,0), 'b': (5,0)}, # Sf4
          HORIZON=4, ORDEM_PARCIAL=[(('d',2,0), ('a',0,1))]),
  2: dict(INITIAL={'c': (0,0), 'd': (3,0), 'a': (0,1), 'b': (1,1)}, # S0
          GOAL   ={'d': (3,0), 'c': (4,1), 'a': (4,2), 'b': (5,2)}, # S5
          HORIZON=5, ORDEM_PARCIAL=[(('c',4,1), ('a',4,2))]),
  3: dict(INITIAL={'c': (0,0), 'a': (3,0), 'b': (5,0), 'd': (3,1)}, # S0
          GOAL   ={'c': (0,0), 'a': (0,1), 'b': (1,1), 'd': (3,0)}, # S7
          HORIZON=6, ORDEM_PARCIAL=[(('b',1,1), ('d',3,0))]),
}
```

2. Rodar os três cenários. Para cada N , dentro da pasta cenarioN:

```
$ python3 ../bw2cnf_var.py --cenario N # gera .cnf e .map
$ minisat trab01_blocos2SAT.cnf resultadoN.txt
$ python3 ../interpretar.py resultadoN.txt --verbose
```

3. Busca do menor horizonte e tamanho das fórmulas. O horizonte de cada cenário é o valor HORIZON no dicionário CENARIOS. Para achar o menor H , começamos com HORIZON=0 e fomos aumentando de um em um. A cada valor geramos a CNF e rodamos o MiniSat, até a primeira resposta SAT. Na Situação 1, por exemplo, com HORIZON=3 o MiniSat responde UNSATISFIABLE e com HORIZON=4 responde SATISFIABLE. Os resultados foram:

Cenário	UNSAT para	SAT com	Variáveis	Cláusulas	Tempo do MiniSat
Sit. 1	$H = 0..3$	$H = 4$	1146	33542	0,0262 s
Sit. 2	$H = 0..4$	$H = 5$	1392	41650	0,0551 s
Sit. 3	$H = 0..5$	$H = 6$	1638	49758	0,0668 s

4. Saída do interpretador. Situação 1:

PLANO ENCONTRADO (4 acoes):

1. t=0: mover bloco 'd' para CIMA de 'c' em p=0
2. t=1: mover bloco 'a' para CIMA de 'b' em p=5
3. t=2: mover bloco 'd' para a MESA em p=2
4. t=3: mover bloco 'a' para CIMA de 'c' em p=0

Situação 2:

PLANO ENCONTRADO (5 acoes):

1. t=0: mover bloco 'b' para CIMA de 'd' em p=3
2. t=1: mover bloco 'a' para a MESA em p=2
3. t=2: mover bloco 'c' para CIMA de 'd' em p=4
4. t=3: mover bloco 'a' para CIMA de 'c' em p=4
5. t=4: mover bloco 'b' para CIMA de 'c' em p=5

Situação 3:

PLANO ENCONTRADO (6 acoes):

1. t=0: mover bloco 'd' para CIMA de 'c' em p=0
2. t=1: mover bloco 'a' para CIMA de 'b' em p=5
3. t=2: mover bloco 'd' para a MESA em p=2
4. t=3: mover bloco 'a' para CIMA de 'c' em p=0
5. t=4: mover bloco 'b' para CIMA de 'c' em p=1
6. t=5: mover bloco 'd' para a MESA em p=3

A saída completa, com o desenho de cada estado, está em `cenarioN/plano_interpretado.txt`.

7 Interpretação da Saída do SAT Solver

7.1 De inteiros para o plano

Vamos explicar usando a Situação 1. Os outros cenários funcionam do mesmo jeito. O `resultado1.txt` tem duas linhas. A primeira é SAT. A segunda tem o valor de cada uma das 1146 variáveis (número positivo quer dizer verdadeiro e negativo quer dizer falso) e termina em 0:

```
SAT
-1 -2 -3 4 -5 -6 -7 -8 -9 -10 -11 12 13 -14 ... 0
```

Só 120 das 1146 variáveis são verdadeiras. O MiniSat só conhece números, então usamos o `.map` para saber o que cada número significa. O `interpretar.py` faz três coisas:

1. lê o mapa, por exemplo $4 \mapsto \text{at}(a, 3, 0)$ e $762 \mapsto \text{move}(d, c, 0, 0)$;
2. fica só com os números positivos;
3. separa as ações e ordena pelo instante t (o último argumento). As ações verdadeiras são 762, 775, 936, $944 \mapsto \text{move}(d, c, 0, 0), \text{move}(a, b, 5, 1), \text{move}(d, T, 2, 2), \text{move}(a, c, 0, 3)$.

Com as variáveis $\text{at}(b, p, t)$ e $\text{lev}(b, l, t)$ verdadeiras dá para montar o estado de cada instante, e o on é calculado a partir delas. Com a opção `--verbose` o programa desenha cada estado:

```
t=0 (estado inicial)                t=4 depois de: mover 'a' para CIMA de 'c'
nível 1 | . . . d d d                nível 1 | a . . . .
nível 0 | c c . a . b                nível 0 | c c d d d b
slots  0 1 2 3 4 5                    slots  0 1 2 3 4 5
(d esta sobre: a, b -> ponte)         a esta sobre: c; b, c, d na MESA
```

Nos outros cenários, as ações verdadeiras são:

- Situação 2 (139 de 1392 variáveis verdadeiras): 862, 927, 1053, 1085, 1194 $\mapsto$ move(b,d,3,0), move(a,T,2,1), move(c,d,4,2), move(a,c,4,3), move(b,c,5,4);
- Situação 3 (168 de 1638): 1036, 1049, 1210, 1218, 1327, 1463 $\mapsto$ move(d,c,0,0), move(a,b,5,1), move(d,T,2,2), move(a,c,0,3), move(b,c,1,4), move(d,T,3,5).

Conferindo pelo terminal. Dá para fazer a mesma tradução sem o `interpretar.py`, só com `grep` e `awk`, do mesmo jeito que no mundo dos blocos clássico. Como as ações aparecem no `.map` com o nome `move`, basta filtrar por essa palavra:

```
$ grep -E "(762|775|936|944)" trab01_blocos2SAT.map
762 move(d,c,0,0)
775 move(a,b,5,1)
936 move(d,T,2,2)
944 move(a,c,0,3)

$ awk 'NR==2 {for(i=1;i<NF;i++) if($i>0) print $i}' resultado1.txt > ids.txt
$ while read id; do grep "^$id " trab01_blocos2SAT.map; done < ids.txt | grep move
```

O segundo comando pega todos os inteiros positivos da linha 2 do resultado, procura cada um no mapa e fica só com as linhas de ação. A saída é a mesma do primeiro.

7.2 Comparação: plano manual $\times$ plano do SAT

Passo	Situação 1 (manual = SAT)	Situação 2: manual	Situação 2: SAT
1	move(d,c,0)	move(b,T,2)	move(b,d,3)
2	move(a,b,5)	move(a,b,2)	move(a,T,2)
3	move(d,T,2)	move(c,d,4)	move(c,d,4)
4	move(a,c,0)	move(a,c,4)	move(a,c,4)
5		move(b,c,5)	move(b,c,5)

- Situação 1: o plano manual e o do SAT são iguais.
- Situação 2: os planos são diferentes, mas os dois têm 5 ações. Existe mais de um plano com 5 ações, e o SAT devolve qualquer um que satisfaça a fórmula. O SAT guardou *b* em cima de *d* e *a* na mesa. O plano manual guarda *b* na mesa e *a* em cima de *b*. O plano do SAT segue a mesma ideia do plano alternativo da Figura 6, só que com os lugares temporários de *a* e *b* trocados. Também testamos obrigar o SAT a usar as 5 ações do plano manual, colocando cada uma como cláusula unitária. A fórmula continuou satisfatível, então o plano manual também é aceito.
- Situação 3: o plano manual e o do SAT são iguais. As ações 1 a 4 são as mesmas da Situação 1.

7.3 Resumo

- Os menores planos têm 4, 5 e 6 ações (Situações 1, 2 e 3). Em cada caso, com uma ação a menos a fórmula é UNSAT.
- Na ordem parcial, os planos das Situações 1 e 3 são cadeias. Na Situação 2 o plano da figura também é uma cadeia, por causa de uma ameaça, mas existe um plano alternativo com 4 linearizações.
- O predicado novo $esp(y, s, t)$ foi necessário. Com o $clr(y)$ do mundo clássico, as metas das Situações 2 e 3 não podem ser alcançadas.

Referências

- [1] S. Russell, P. Norvig. *Inteligência Artificial: Uma Abordagem Moderna*. Capítulos de planejamento clássico e planejamento de ordem parcial.
- [2] I. Bratko. *Prolog Programming for Artificial Intelligence*. Capítulo de planejamento.
- [3] H. Kautz, B. Selman. Planning as Satisfiability. *ECAI*, 1992.
- [4] N. Eén, N. Sörensson. An Extensible SAT-solver. *SAT*, 2003.